HENJAI

เห็นใจ

คู่มือการพัฒนาระบบ
System Manual

โครงการผู้ช่วยบรรยายภาพด้วยปัญญาประดิษฐ์
ที่เข้าใจอารมณ์และบริบทสำหรับผู้พิการทางสายตา

Emotion & Context-Aware AI Vision Assistant

ผู้จัดทำ

นายกิตติพงศ์ สมสอาด
นายชินพัฒน์ ขุนเจริญ
นายเรข์ณพัศ สุวรรณปริสุทธิ์

ทีม Rushhour
นิสิตชั้นปีที่ 2 หลักสูตรวิศวกรรมคอมพิวเตอร์และเทคโนโลยีดิจิทัล (CEDT)
ภาควิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย

ขอบเขตและสถานะการส่งมอบ

คู่มือ นี้ สำหรับ ผู้รับ ช่วง ซอร์ส HENJAI ครอบคลุม โครงสร้าง โปรแกรม การ ติด ตั้ง การ ตั้ง ค่า สัญญา ข้อมูล การ ประเมิน และ การ ดูแล ระบบ อ้างอิง ไฟล์ จริง ใน โครงการ และ snapshot ซอร์ส commit `e327cd1dd1f2f8aaaea47709fa080b78a309bdea` พร้อมบัญชี SHA-256 ของไฟล์ส่งมอบ

ไม่ได้เปลี่ยนซอฟต์แวร์ต้นแบบหรือรันโมเดลภายนอกใหม่เพื่อจัดทำเอกสาร Dependency ของโครงการ ยังไม่ได้ตรึงเวอร์ชันทั้งหมด และรุ่นโมเดลในโค้ดต้องตรวจความพร้อมกับบัญชีบริการก่อนใช้งานจริง คู่มือนี้จึง แยกขั้นตอนติดตั้งที่แนะนำออกจากผลการตรวจที่ทำแล้วใน `VALIDATION.md`

สารบัญ

ขอบเขตและสถานะการส่งมอบ **i**

1 โครงสร้างและองค์ประกอบ **1**

1.1 ไฟล์หลัก 1

1.2 ขอบเขต snapshot 1

2 ติดตั้งและตั้งค่าต้นแบบ Python **2**

2.1 เตรียมสภาพแวดล้อม 2

2.2 API key และโมเดล 2

2.3 เตรียมโมเดลในเครื่อง 3

2.4 เริ่มบริการและตรวจขั้นต้น 3

3 ติดตั้งและดูแลต้นแบบเว็บ **4**

3.1 องค์ประกอบและตัวแปรแวดล้อม 4

3.2 การทดสอบในเครื่องและการเผยแพร่ 4

4 สัญญาข้อมูลและจุดปรับระบบ **5**

4.1 API ของ Flask 5

4.2 API ของเว็บ 5

4.3 ค่าปรับที่มีผลต่อการทดลอง 6

4.4 เสียงและความต่างระหว่างแพลตฟอร์ม 6

5 การทดสอบ การประเมิน และการบำรุงรักษา **8**

5.1 ทดสอบกฎโดยไม่เรียกโมเดล 8

5.2 นำข้อมูลส่งมอบกลับเข้าโฟลเดอร์ทดสอบ 8

5.3 รันการวิเคราะห์ใหม่อย่างควบคุม 8

5.4 การให้คะแนนและสรุปผล 9
5.5 ข้อมูล ความลับ และการสำรอง 9
5.6 ข้อจำกัดที่ต้องแก้ก่อนเปิดสาธารณะ 9

A การคอมไพล์เอกสาร LaTeX 10

บทที่ 1

โครงสร้างและองค์ประกอบ

1.1 ไฟล์หลัก

ตารางที่ 1.1: แผนที่ซอร์สสำหรับผู้รับช่วง

ไฟล์หรือโฟลเดอร์	หน้าที่
`poc/server.py`	Flask routes หน้าจอ สถานะ การตั้งค่า และ MJPEG
`poc/core.py`	กล้อง Session การเปลี่ยนภาพ Gemini ความจำ และ TTS
`poc/fast.py`	YOLO/Depth/MediaPipe และกฎเตือนระยะชิด
`poc/memory.py`	เก็บประโยคล่าสุด จัด prompt บริบท และตรวจซ้ำ
`poc/templates/`	หน้าจอผู้ใช้กับแดชบอร์ดพัฒนา
`poc/models/`	ไฟล์ MediaPipe face landmarker ที่มีในโครงการ
`poc/*.pt`	น้ำหนัก YOLO ที่มีในโครงการ
`poc/eval/`	สคริปต์เก็บภาพ เลือกภาพ รัน และเปรียบเทียบโมเดล
`web/public/index.html`	กล้อง canvas UI ความจำ และ TTS ในเบราว์เซอร์
`web/api/analyze.js`	ฟังก์ชัน serverless เรียก Gemini และส่งผล JSON
`web/vercel.json`	region และ maxDuration ของฟังก์ชัน
`web/package.json`	ประเภท module และ Node.js ขั้นต่ำ

ต้นแบบปัจจุบันใช้ Flask ไม่ใช่ Streamlit แม้เอกสาร handoff รุ่นเก่ายังกล่าวถึง `app.py` ผู้รับช่วงควรเริ่มจาก `server.py` และ `core.py` โค้ดใน `bench_local.py` เป็นการทดลอง FER แยก ไม่ใช่รายการ dependency ครบของ `fast.py` ปัจจุบัน

1.2 ขอบเขต snapshot

`source_files/` เก็บไฟล์โปรแกรมจาก `poc/` และ `web/` ที่จำเป็น พร้อมโมเดลในโครงการและเอกสารอ้างอิงภายในที่เกี่ยวข้อง ส่วนภาพกับผลการทดลองอยู่ใน `data/` แยกจากซอร์สเพื่อไม่ให้การรันทดสอบเขียนทับหลักฐานส่งมอบ

บทที่ 2

ติดตั้งและตั้งค่าต้นแบบ Python

2.1 เตรียมสภาพแวดล้อม

ฐานที่เหมาะกับต้นแบบในโครงการคือ macOS ที่มี Python และกล้อง รองรับ MPS จะช่วยการประมวลผลบางส่วน แต่โค้ดมีทางเลือก CPU ด้วย เสียง `say`, `afplay` และ voice Kanya ผูกกับ macOS การใช้ระบบอื่นต้องปรับ backend เสียงและตรวจใหม่

ตัวอย่างสร้าง environment จากโฟลเดอร์ `delivery/`:

```
python3 -m venv .venv-henjai
source .venv-henjai/bin/activate
python -m pip install -r source_files/poc/requirements.txt
cd source_files/poc
```

ใช้ Python ตัวเดียวกันติดตั้งและรันโปรแกรม ไฟล์ requirements เดิมมี Pillow, dotenv, Google SDK, gTTS, Ultralytics, OpenCV, NumPy และ Flask แต่ไม่ได้ระบุ dependency สายเร็วครบทั้งหมด จาก imports ปัจจุบันต้องมี Torch, Transformers และ MediaPipe ด้วย ตรวจ environment ก่อนเปิด fast

```
python -m pip install torch transformers mediapipe
python -c "import flask, cv2, numpy, torch, transformers, mediapipe"
```

คำสั่งเพิ่ม dependency เป็นแนวทางเติมสิ่งที่ import จริง ยังไม่ได้ยืนยันชุดเวอร์ชันที่เข้ากันได้บนเครื่องใหม่ ควรสร้าง environment แยกและบันทึก `pipfreeze` เมื่อทดสอบผ่านแล้ว โดยเฉพาะเมื่อ Python หรือแพ็กเกจหลักเปลี่ยนรุ่น

2.2 API key และโมเดล

สร้างไฟล์ `.env` ใน `source_files/poc/` ด้วยข้อความตัวอย่างต่อไปนี้ โดยใส่ key ของบัญชีที่ได้รับอนุญาต

```
GEMINI_API_KEY=YOUR_KEY_HERE
```

โค้ดเรียก `load_dotenv()` และอ่านค่า key ใน session ไม่ต้องมีไฟล์ `.env.example` จึงจะทำงานได้ ให้จำกัดสิทธิ์อ่านไฟล์และไม่ commit ไฟล์นี้ รายการโมเดลที่เลือกได้อยู่ใน `core.MODELS` โดยค่าต้น

รายการเป็น `gemini-3.1-flash-lite` ตามซอร์สเดิม รายชื่อนี้ไม่ใช่การรับรองว่าผู้ให้บริการยังเปิดให้บัญชีทุกประเภทใช้งาน

หากชื่อโมเดลใช้ไม่ได้ ให้ตรวจรุ่นที่บัญชีรองรับแล้วปรับ `MODELS` หรือค่า model ของ session ผ่านแดชบอร์ด การเปลี่ยนรุ่นโมเดลถือเป็นเงื่อนไขการทดลองใหม่ ต้องบันทึกแยกจากผลเดิม

2.3 เตรียมโมเดลในเครื่อง

ไฟล์ที่มีในชุดซอร์สได้แก่ `yolo11n.pt`, `yolo11n-pose.pt` และ `models/face_landmarker.task` โค้ดอ้างพาธสัมพันธ์กับ working directory จึงต้องรันจาก `poc/` สำหรับความลึก โค้ดโหลด `depth-anything/Depth-Anything-V2-Metric-Indoor-Small-hf` ผ่าน Transformers; การโหลดครั้งแรกอาจต้องใช้อินเทอร์เน็ตและพื้นที่ cache

โมเดล FER `trpakov/vit-face-expression` เป็นของสคริปต์ benchmark แยก และอาจดาวน์โหลดเพิ่มเมื่อต้องการรัน benchmark นั้น การเรียกสายเร็วโดยไม่ใช้ cloud ยังต้องมีไฟล์โมเดลครบก่อน จึงไม่ควรโฆษณาว่าพร้อมทำงาน offline ตั้งแต่ติดตั้งใหม่

2.4 เริ่มบริการและตรวจขั้นต้น

```
python server.py
```

เปิด `http://127.0.0.1:8000/dev` เลือกกล้องและโมเดล แล้วทดสอบเริ่มและหยุด เปิดหน้าผู้ใช้ที่ `http://127.0.0.1:8000/` บริการผูกกับ loopback และ `debug=False` ตามซอร์ส ไม่เปิดรับจากเครือข่ายภายนอกโดยเปลี่ยน host ก่อนจัดการสิทธิ์และการป้องกัน API

บทที่ 3

ติดตั้งและดูแลต้นแบบเว็บ

3.1 องค์ประกอบและตัวแปรแวดล้อม

โปรเจกต์เว็บกำหนด Node.js ตั้งแต่รุ่น 20 และ `type=module` ไม่มีขั้นตอน build framework เพิ่ม Frontend อยู่ใน `public/index.html` และ API อยู่ใน `api/analyze.js` ตั้งค่า region เป็น `sin1` และ maxDuration 30 วินาที ขณะที่ตัวเรียก Gemini มี timeout 25 วินาที

ตารางที่ 3.1: ตัวแปรฝั่งเซิร์ฟเวอร์ของเว็บ

ตัวแปร	ความจำเป็น	ความหมาย
`GEMINI_API_KEY`	ต้องมี	key ที่ serverless ใช้เรียกบริการ
`GEMINI_MODEL`	ไม่บังคับ	ถ้าไม่ตั้ง ใช้ gemini-3.1-flash-lite ตามโค้ด

ไม่ใส่ key ใน HTML, JavaScript ที่ส่งให้เบราว์เซอร์ หรือ URL หน้าเว็บ ผู้รับช่วงควรตั้ง environment ในแพลตฟอร์มโฮสต์ แล้ว deploy ใหม่ตามขั้นตอนของตน

3.2 การทดสอบในเครื่องและการเผยแพร่

หากมี Vercel CLI ให้เปิด terminal ใน `source_files/web/` แล้วใช้

```
vercel dev
```

ต้องมี key ใน environment ที่ CLI อ่านได้ สำหรับการเตรียม preview ใช้ `vercel` และตรวจ URL ที่ได้จริงก่อนเผยแพร่ production คู่มือนี้ไม่ได้สั่ง deploy หรืออ้างว่ามี deployment ใหม่สำเร็จแล้ว

ทดสอบผ่าน localhost หรือ HTTPS ให้กล้องทำงานตามนโยบายเบราว์เซอร์ จากนั้นตรวจเสียงไทย หน้าแนวตั้ง และการกดเริ่มหยุด เพิ่ม `?dev=1` เพื่อดูสถานะรอบวิเคราะห์ เว็บไม่มี local YOLO/Depth จึงไม่ควรใช้ภาพหน้าจอเว็บยืนยันว่าตรวจระยะในเครื่องทำงาน

บทที่ 4

สัญญาข้อมูลและจุดปรับระบบ

4.1 API ของ Flask

ตารางที่ 4.1: เส้นทางบริการต้นแบบ Python

Path	Method	หน้าที่
`/`	GET	หน้าจอผู้ใช้
`/dev`	GET	แดชบอร์ดกล้อง โมเดล และสถานะ
`/state`	GET	JSON สถานะ session
`/toggle`	POST	เริ่มหรือหยุดตามสถานะปัจจุบัน
`/config`	POST	JSON ตั้งค่า cam, every, model, mirror, still, scene, alert, fast_on และคำสั่ง forget/recalibrate
`/video`	GET	MJPEG; เติม `?dev=1` สำหรับภาพตรวจสอบ

`/config` ยังไม่ได้ validate ค่าทุกฟิลด์อย่างเข้มงวด การส่งค่าผิดประเภทหรือเกณฑ์ไม่เรียงกันอาจทำให้พฤติกรรมไม่ถูกต้อง ใช้ผ่านแดชบอร์ดในเครื่องที่ควบคุมได้ก่อน หากต้องเปิด API ให้ภายนอกต้องเพิ่มการตรวจข้อมูลและสิทธิ์

4.2 API ของเว็บ

`POST/api/analyze` รับ JSON ที่มี image เป็น base64 JPEG, recent เป็นรายการคำบรรยาย, summary เป็นสรุปเดิม และ mode เป็น first/delta/scene/alert ตัวอย่างเชิงโครงสร้าง:

```
{
  "image": "<base64 JPEG without data URL prefix>",
  "recent": [],
  "summary": "",
  "mode": "first"
```

}

ฟิลด์ หลัก ใน ผล สำเร็จ คือ `narrative_th`, `reading`, `emotion`, `body_language`, `environment`, `memory_th` และ `_ms` ซึ่ง เป็น เวลา ของ handler หาก แปลง JSON โมเดล ไม่ ได้ จะ fallback เป็นคำบรรยายและ `_raw` โดยไม่ได้ตรวจ schema เพิ่ม

ตารางที่ 4.2: สถานะ HTTP และการจัดการใน API เว็บ

สถานะ	สาเหตุใน handler
200	ได้ผลหรือ fallback จากคำตอบโมเดล
400	ไม่ส่ง image
405	ใช้ method อื่นแทน POST
500	ไม่ตั้ง API key
รหัสจาก upstream	บริการ Gemini ปฏิเสธคำขอ เช่น โควตา
504	timeout หรือ exception ในบล็อกเรียกบริการ

4.3 ค่าปรับที่มีผลต่อการทดลอง

- `SYSTEM_PROMPT` และ `MODE_PROMPT`: ควบคุมเนื้อหา การอ่านเอกสาร และการเล่าสิ่งเปลี่ยน; มีสำเนาแยกใน Python กับเว็บ ต้องตรวจให้สอดคล้องเมื่อตั้งใจเปลี่ยนทั้งสองระบบ
- `MEM_RECENT=5`, `MEM_CAP=600`: จำกัดประวัติและสรุป; ต้องตรวจโค้ดเว็บแยกเมื่อเปลี่ยน
- Python `NOISE_MULT=(2.5,9,22)` และ เว็บ `STILL=.02,SCENE=.18,ALERT=.45`: กลไกต่างกัน ไม่ควรอ้างว่าใช้ threshold เดียวกัน
- `STOP_M=.5`, `RELEASE_M=.9`, `WARN_M=1.5`: ค่า กฎ ระยะ ใน เครื่อง ต้อง สอบ เทียบ ใหม่ หาก เปลี่ยน กล้อง หรือ โมเดล
- `temperature=.4`, `max_output_tokens=512`, timeout 25 วินาที: เปลี่ยนแล้วต้องบันทึกเป็นเงื่อนไขใหม่

4.4 เสียงและความต่างระหว่างแพลตฟอร์ม

Python สายคลาวด์ใช้ gTTS ก่อน แล้ว fallback เป็น macOS say ที่สร้าง AIFF และเล่นด้วย afplay สายเร็วใช้ say พูดโดยตรง เว็บเลือกเสียงภาษาไทยจาก speechSynthesis และแบ่งข้อความเป็นช่วง การกันพูดซ้ำ

ของ Python ใช้ SequenceMatcher แต่เว็บใช้ชุดคำแยกด้วยช่องว่าง ต้องทดสอบพฤติกรรมภาษาไทยทั้งสองเส้นทางแยกกัน

บทที่ 5

การทดสอบ การประเมิน และการบำรุงรักษา

5.1 ทดสอบกฎโดยไม่เรียกโมเดล

จากโฟลเดอร์ `source_files/poc/` ใช้คำสั่ง

```
python test_memory.py
python test_approach.py
```

ชุดแรกตรวจประวัติ ความคล้ายข้อความ คะแนนภาพต่าง และเกณฑ์จาก noise ชุดที่สองตรวจเตือนเมื่อชิด กันเตือนซ้ำ และเปิดเตือนใหม่เมื่อถอยพ้นระยะ ชุดทดสอบหลัง import `fast.py` จึงต้องมี Torch/OpenCV/NumPy แต่ไม่เรียก `_models()` และไม่จำเป็นต้องดาวน์โหลดน้ำหนักใหม่

ผลผ่านของกฎไม่ยืนยันว่าระยะที่โมเดลวัดได้ถูกต้อง และไม่ยืนยันเสียงจะทันเหตุการณ์จริง ดูผลการตรวจในรอบส่งมอบจาก `VALIDATION.md` ซึ่งระบุ environment และข้อจำกัดแยกไว้

5.2 นำข้อมูลส่งมอบกลับเข้าโฟลเดอร์ทดสอบ

ไฟล์ข้อมูลส่งมอบอยู่ใน `data/` ส่วนสคริปต์เดิมคาดว่าจะพบใน `poc/eval/` ให้สร้างสำเนาทดลองจากซอร์สก่อน แล้วคัดลอก `data/img/`, `manifest.json`, `results.json`, `results_before.json`, `score.csv` และ `picks.json` ไปยัง `poc/eval/` ของสำเนานั้น อย่าใช้โฟลเดอร์หลักฐานต้นฉบับเป็นที่เขียนผลใหม่

จาก `poc/` ของสำเนา ใช้ `pythoneval/score.py` เพื่อสรุปคะแนนที่กรอกแล้ว โดยไฟล์ส่งมอบเดิมยังไม่มีคะแนนมนุษย์ สคริปต์จึงควรแจ้งว่ายังไม่ได้กรอก ไม่ควรแปลงค่าว่างเป็นศูนย์หรือสมมติคะแนนขึ้นเอง

5.3 รันการวิเคราะห์ใหม่อย่างควบคุม

`pythoneval/run.py` เรียก Gemini ตาม manifest และเขียนทับทั้งผลกับตารางให้คะแนน ส่วน `pythoneval/run.pyemotion` จำกัดกลุ่มได้ การรันนี้ใช้บริการภายนอกและโควตา ต้องตรึงชุดภาพ model prompt และค่าทดลอง รวมทั้งสำรองผลก่อนทุกครั้ง

ชุดข้อมูลปัจจุบันมี 25 ภาพ ขณะที่ผลเดิมมี 24 รายการ การรันใหม่ทั้ง manifest จึงไม่ใช่การทำซ้ำด้วยจำนวนภาพเดิมทันที ต้องกำหนดรายการทดลองอย่างชัดเจน อีกทั้งภาพ emotion_6 ยังมีสถานะสิทธิ์ที่ต้องตรวจ

5.4 การให้คะแนนและสรุปผล

ให้ผู้ประเมินเปิดภาพและอ่านคำบรรยายโดยไม่ใช้คำค้นเป็นคำตอบอ้างอิงโดยอัตโนมัติ correctness ใช้ 1 เมื่อใจความสำคัญถูกต้องตามภาพ helpfulness กับ conciseness ให้ 1–5 ตามเกณฑ์ที่ตกลงร่วมกัน และ hallucination ใช้ 1 เมื่อมีเนื้อหาที่ไม่มีหลักฐานในภาพ ต้องกำหนดแนวทางภาพกำกวมและความเห็นไม่ตรงกันก่อนเริ่ม

รายงานทั้งจำนวนที่ประมวลผลสำเร็จ จำนวนที่ให้คะแนนแล้ว และจำนวนที่ยังขาด แยกความผิดพลาด API ออกจากความผิดพลาดความหมาย หากเทียบคลาสอารมณ์ ให้ปรับ alias เช่น surprise/surprised อย่างเปิดเผย ตัวเลข 4/6 จากการเทียบข้อความตรงตัวต่างจาก 5/6 เมื่อปรับ alias และทั้งคู่ยังอาศัยป้ายชั่วคราว

5.5 ข้อมูล ความลับ และการสำรอง

เก็บ key แยกจากซอร์ส จำกัดผู้เข้าถึงภาพที่มีบุคคลหรือข้อมูลส่วนตัว และกำหนดระยะเก็บไฟล์เสียงชั่วคราวที่เขียนใน `/tmp/` เมื่อจะใช้ระยะยาว ตรวจเงื่อนไขผู้ให้บริการ AI/TTS และทรัพยากรบุคคลที่สามด้วย อย่าอนุมานว่าการใช้ local model ทำให้ทุกเส้นทางของระบบไม่ส่งข้อมูลภายนอก

สำรองผลดิบและ CSV ก่อนรันซ้ำ ใช้ชื่อโฟลเดอร์รอบทดลองพร้อมวันที่และบันทึก config เปรียบเทียบ SHA-256 กับบัญชีส่งมอบเมื่อต้องยืนยันไฟล์ต้นฉบับ หากแก้โปรแกรมให้เก็บ diff และผลทดสอบกฎที่เกี่ยวข้องไว้ด้วย

5.6 ข้อจำกัดที่ต้องแก้ก่อนเปิดสาธารณะ

ต้นแบบยังไม่มีการยืนยันตัวตน การจำกัดคำขอต่อผู้ใช้ และการตรวจ schema ครบถ้วน API เว็บรับภาพและบริบทจากผู้เรียก จึงต้องกำหนดขนาด รูปแบบ และขอบเขตข้อมูลก่อนเปิดให้ผู้ใช้ทั่วไป นอกจากนี้ควรเพิ่มการติดตามความผิดพลาดอย่างไม่บันทึกข้อมูลอ่อนไหว และทดสอบการหยุดระหว่างคำขอหรือเสียงที่ค้าง

ให้รับช่วงจากระบบที่เล็กและตรวจได้ก่อน เมื่อเพิ่มโมเดล เปลี่ยน threshold หรือเพิ่มฟังก์ชัน ต้องระบุผลต่อเวลา โควตา การส่งข้อมูล และกฎเสียง ไม่ใช้ตัวเลข benchmark เก่ากับระบบที่มีองค์ประกอบเปลี่ยนแล้ว

ภาคผนวก A

การคอมไพล์เอกสาร LaTeX

ไฟล์ `latex/` มีต้นฉบับทั้งสามเล่ม ใช้ Tectonic หรือ XeLaTeX และแนบฟอนต์ TH SarabunPSK ของ SIPA เพื่อรองรับภาษาไทย (เนื้อหา 16 pt หัวเรื่อง 18 pt) คอมไพล์จากโฟลเดอร์ดังกล่าวด้วย

```
bash build.sh
```

สคริปต์ส่ง PDF ไปยัง `../pdf/` หากใช้ XeLaTeX ให้รันอย่างน้อยสามรอบเพื่ออัปเดตสารบัญ การอ้างอิง และหมายเลขหน้า โดยใช้ `-output-directory=../pdf` กับแต่ละไฟล์ ฟอนต์และภาพประกอบอ้างพาธจาก working directory `latex/` จึงไม่ควรรันจากโฟลเดอร์อื่นโดยไม่ปรับพาธ

Tectonic อาจดาวน์โหลดแพ็กเกจ TeX ในการคอมไพล์ครั้งแรก เมื่อแก้ข้อมูลในรายงานต้องตรวจ PDF ที่ได้อีกครั้ง โดยเฉพาะชื่อบุคคล ตารางภาษาไทย สารบัญ และ URL ที่ยาว เอกสารเปลี่ยนแล้วควรปรับบัญชี checksum ก่อนส่งมอบรอบใหม่